

PEC 2 - Visión Artificial

Introducción

El procesamiento digital de imágenes representa uno de los pilares fundamentales de la visión artificial, permitiendo analizar, transformar y extraer información útil a partir de datos visuales. En esta primera memoria de la asignatura, se recoge el trabajo realizado en la PEC 2 de Visión Artificial, donde el objetivo principal es avanzar con un conjunto amplio y representativo de técnicas esenciales de la librería OpenCV.

A lo largo del desarrollo de esta PEC se han abordado problemas reales y variados: detección de letras, orientaciones, fusión de imágenes por diferentes perspectivas, detección por cámara de caras y cuerpos y clasificación de cruces en una misma imagen con diferentes complicaciones cada una. Cada uno de estos bloques introduce conceptos diferentes en su mayoría facilitados en los tutoriales de la librería principal.

Siguiendo con la línea, el trabajo se ha implementado íntegramente en Python utilizando la librería OpenCV, lo que ha permitido aplicar toda la manipulación de imágenes en base a las guías de la misma. Igualmente la idea no era replicar los tutoriales, sino adaptarlos para problemas reales.

En forma de preludio, toda esta entrega permite recorrer un espectro completo que va desde el preprocesado básico hasta métodos avanzados de segmentación y reconstrucción geométrica. Y esta memoria se organiza de forma metódica, unidireccional y coherente, integrando análisis, justificación técnica y resultados visuales, con el objetivo de demostrar la comprensión de los métodos aplicados y su relevancia en distintos problemas de visión artificial.

Ejercicio 1

Este ejercicio se basa en el conocimiento geométrico del dominio (orientación global y líneas dominantes) con la doble finalidad de estimar la dirección del texto para su posterior normalización mediante una rotación, y de ajustar un modelo lineal robusto frente a ruido extremo, comparando dos metodologías (RANSAC y la Transformada de Hough), siguiendo los tutoriales y apartados especificados en el enunciado.

Ejercicio 1.a

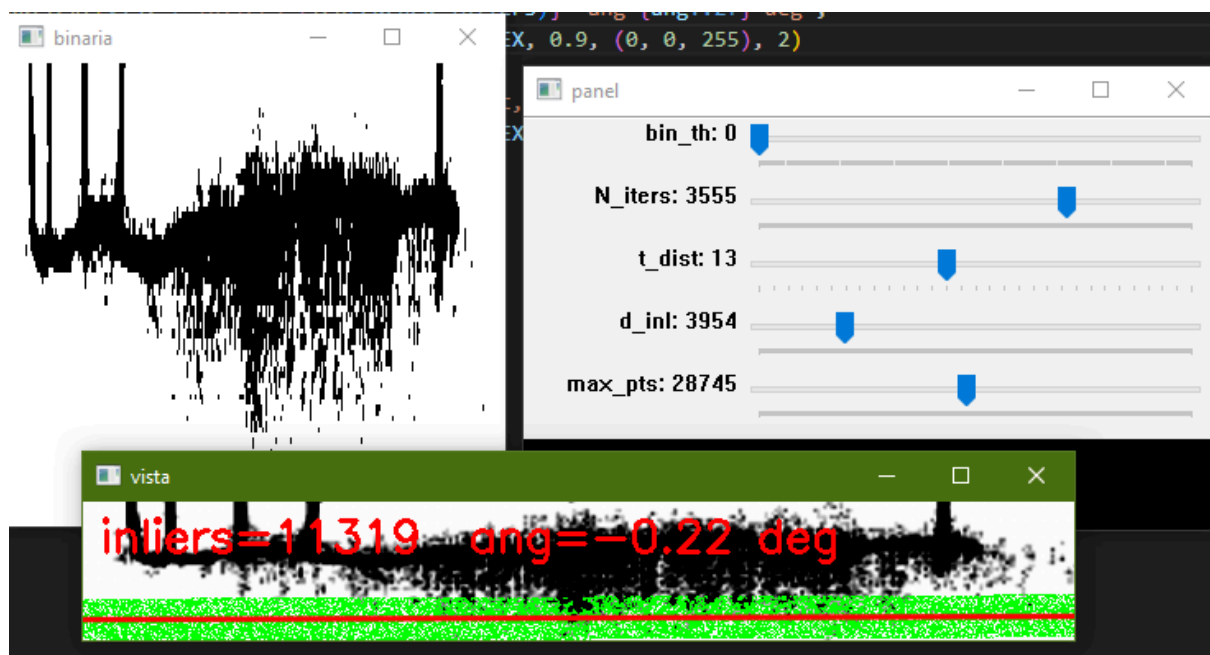
El proceso busca enderezar y acondicionar un texto inclinado de la imagen textoMolinos.png para su análisis. La metodología incluye: preprocesamiento (escala de grises, Canny), estimación del ángulo de inclinación dominante usando la mediana de los ángulos de Hough, corrección afín (warpAffine) con fondo blanco, binarización (manual u Otsu) e inversión. Sigue la segmentación de componentes conexos, filtrando por área. La fase final es la normalización de cada carácter a 20x20 píxeles, con dos modos: (a) estiramiento directo (con posible distorsión) o (b) preservación de aspecto (con padding). Se genera una visualización de las fases clave. El rendimiento depende de ajustes críticos como el Umbral de Hough, el Umbral de Binarización, los Límites de Área y la estrategia de normalización. Los desafíos incluyen la fiabilidad de la estimación de Hough, la dependencia crítica del umbral de binarización (que puede causar fragmentación o fusión de caracteres) y la distorsión del estiramiento o la variabilidad del padding en la normalización.



La estimación de inclinación por Hough se estabiliza en 8°, y con un offset manual se logra una corrección final de 17, enderezando el texto. La binarización y el filtrado por área (min_area=23, max_area=152) segmentan caracteres individuales, y la normalización 20x20 con preservación de aspecto (norm_mode=1) genera un mosaico de letras consistente, aunque con pequeñas variaciones por fragmentación (ej: signos o letras estrechas y muy juntas). Esta normalización es útil porque convierte cada letra en una entrada de tamaño fijo, facilitando el tratamiento automático posterior. Entre las dos opciones, la opción 2 suele ser preferible para no introducir distorsión geométrica, aunque añade padding y variabilidad de ocupación; la opción 1 homogeneiza más el tamaño ocupado, pero puede deformar letras.

Ejercicio 1.b

En este ejercicio se pide estimar la línea principal en `lineaRuidosa.png` en presencia de una gran cantidad de ruido y outliers, utilizando un ajuste robusto mediante RANSAC. El procedimiento aplicado consiste en binarizar la imagen para obtener un conjunto de puntos candidatos (píxeles activos) y, si la nube es demasiado grande, limitar el coste computacional mediante submuestreo/control del número máximo de puntos. A partir de ahí, RANSAC itera: selecciona dos puntos aleatorios, define una recta candidata, calcula la distancia de todos los puntos a la recta y marca como inliers aquellos que caen por debajo de un umbral t_dist ; el mejor modelo se elige por máximo número de inliers (y/o cumplimiento de un mínimo d_inl) y se visualiza la recta final junto con el conjunto de inliers para validar estabilidad frente a ruido.



En la configuración mostrada, con $N_iters=3555$ y $t_dist=13$, el algoritmo encuentra un modelo con $inliers=11319$ y un ángulo $=-0.22^\circ$, es decir, una línea prácticamente horizontal y coherente con la estructura dominante del patrón. La visualización confirma que la recta estimada atraviesa la banda principal, y el número elevado de inliers indica un ajuste estable pese a la fuerte contaminación; aun así, la máscara binaria sigue siendo muy densa (mucho ruido), por lo que el ajuste depende de mantener suficiente número de iteraciones y un t_dist acorde al grosor efectivo de la banda para no perder inliers ni admitir demasiados outliers.

Ejercicio 1.c

En este apartado se vuelve a estimar la línea principal de `lineaRuidosa.png`, pero ahora mediante Transformada de Hough y comparando dos entradas para Hough: una máscara binaria obtenida por umbral global (sin Canny) y un mapa de bordes obtenido con Canny (con un ligero refuerzo morfológico si aplica) (con Canny). La idea es comprobar si Canny es imprescindible: en términos

prácticos, Hough no “necesita” Canny como tal, sino un conjunto de puntos que representen la estructura lineal; ese conjunto puede provenir de bordes más finos y selectivos o de una binaria más densa y sensible a ruido. Por eso, además de estimar el ángulo dominante a partir de las líneas detectadas, se observa cómo cambia la estabilidad del resultado con ambos modos.



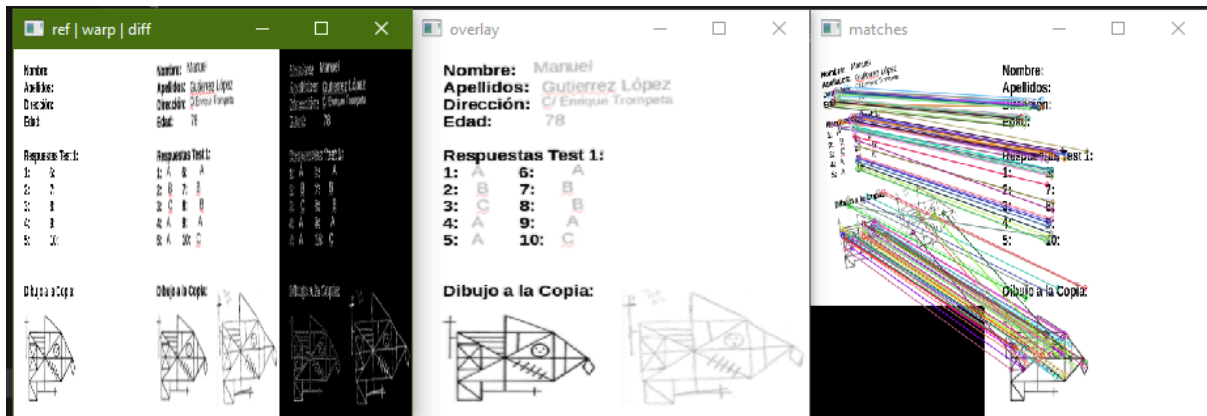
En la comparación mostrada, el modo Sin Canny produce un ángulo estimado de -2.5° , mientras que el modo Con Canny se acerca más a horizontal con -0.5° . Esto encaja con lo esperable: al trabajar sobre bordes, el voto de Hough se concentra en una estructura más limpia y suele reducir la influencia del ruido disperso de la binaria, dando una estimación más estable. Aun así, el hecho de que sin Canny detecte igualmente una línea dominante confirma que Canny no es estrictamente necesario, aunque sí puede mejorar la robustez cuando la binarización deja demasiados puntos irrelevantes.

Ejercicio 2

En este bloque se emplean detectores y descriptores locales para (a) alinear un documento mediante homografía y (b) crear una panorámica por stitching. Basado en los tutoriales indicados en el enunciado.

Ejercicio 2.a

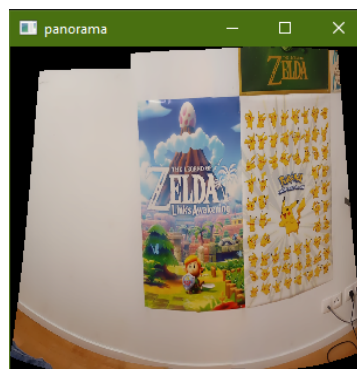
En este ejercicio se alinea un formulario relleno (formularioRelleno-original.png) con un patrón (formulario.png) usando puntos característicos. El flujo implementado es: conversión a gris y preprocesado seleccionable (PREPROC_MODO: sin prepro, CLAHE o Canny), detección y descripción con ORB, emparejamiento con BFMatcher y filtrado por ratio test (RATIO), y estimación de la transformación mediante una homografía robusta con RANSAC, calculada a partir de los matches (función homografia, que internamente usa `cv.findHomography(..., cv.RANSAC, RANSAC_THR)`). Con la homografía se aplica `warpPerspective` para llevar el formulario relleno al sistema del patrón, se guarda el resultado (data/formularioRelleno-alineado.png) y se valida visualmente con el panel ref | warp | diff, un overlay con alpha (ALPHA) y una ventana de matches.



En la imagen se aprecia que el warp queda bien registrado respecto al ref: la estructura global del formulario (bloques y líneas) coincide y el diff es bajo en la mayor parte del documento, quedando las discrepancias concentradas en zonas manuscritas o detalles locales del escaneo. El overlay confirma la alineación global y la visualización de matches muestra correspondencias distribuidas por el documento, lo que justifica que RANSAC encuentre una homografía estable pese a la presencia de matches erróneos.

Ejercicio 2.b

En este ejercicio se construye una panorámica a partir de tres imágenes (poster0.jpeg, poster1.jpeg, poster2.jpeg) utilizando la API de alto nivel de OpenCV. La implementación es directa: se cargan las tres imágenes, se crean como lista y se usa `cv.Stitcher_create(cv.Stitcher_PANORAMA)` seguido de `stitcher.stitch(imagenes)`, obteniendo status y la imagen resultante pano, que se muestra en una ventana. La parte “compleja” (detección de features, matching, estimación de transformaciones, warping y blending) queda encapsulada dentro del Stitcher, por lo que aquí el punto clave es que exista suficiente solape y consistencia geométrica entre vistas.



La panorámica resultante es coherente en la zona central (el póster queda integrado), pero aparecen artefactos típicos en bordes (zonas negras/recortes y cierta deformación), lo que indica que el stitching ha encontrado matches suficientes pero con solape limitado en algunas regiones. Respecto al enunciado, el orden puede importar en la práctica: aunque Stitcher intenta construir el registro de forma global, el orden de entrada puede influir en heurísticas internas (elección de imagen base,

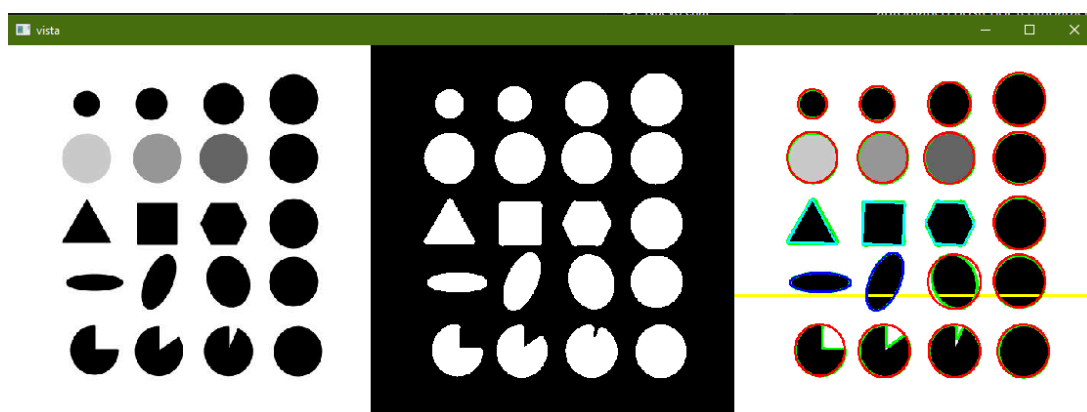
emparejamientos iniciales) cuando hay ambigüedad o poco solape; si el resultado es inestable, conviene probar permutaciones o asegurar un solape más homogéneo entre imágenes consecutivas. En nuestro caso no resulta afectado por el orden.

Ejercicio 3

Se trabajan tres enfoques: (a) análisis de blobs por métricas geométricas e intensidad, (b) detección de caras con Haar cascades, y (c) detección de peatones con HoG + SVM.

Ejercicio 3.a

En este ejercicio se analiza blob.jpg segmentando cada objeto y extrayendo métricas para clasificar forma (círculo/elipse/otro), tamaño (S/M/L), color en escala de grises y, en la última fila, estimar el porcentaje de cuña faltante tipo pacman y el número de lados rectos. El pipeline implementado parte de una segmentación robusta del foreground: se estima el nivel de fondo con la mediana de intensidades en el borde y se fija un umbral relativo $\text{thr} = \text{bg_med} - 10$, se aplica GaussianBlur y binarización invertida, y se limpia con morfología (OPEN/CLOSE) para consolidar blobs. Luego se extraen componentes conectados y contornos, se calcula circularidad $4\pi A/P^2$, solidity (área / área del convex hull) y axis ratio (fitEllipse) para decidir círculo/elipse/otro con umbrales. La última fila se identifica por percentil en Y (LAST_ROW_PERCENTILE), y ahí se estima cuña% comparando el área del contorno con el área del círculo envolvente (minEnclosingCircle) y se fijan 2 lados rectos; para el resto de otros se estima el número de lados con approxPolyDP (vértices como proxy). Finalmente se asigna tamaño por percentiles de área y color por la media del canal V en HSV discretizada en rangos (negro/gris_*), y se dibuja un overlay con contornos/círculos/elipses/polígonos y la línea amarilla de separación de última fila.

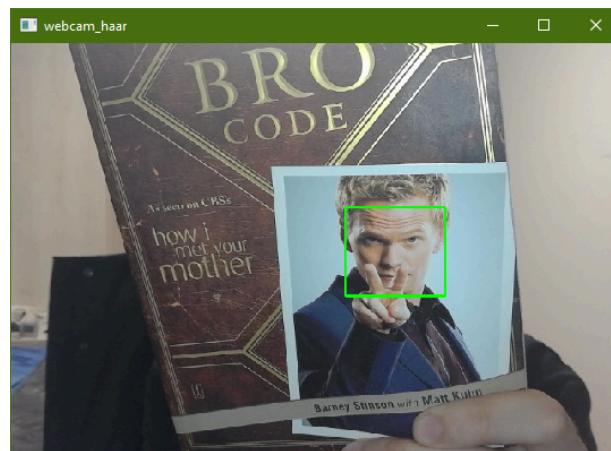


En el resultado se ve que la segmentación separa correctamente los blobs y el overlay marca círculos (círculo envolvente en rojo), elipses (elipse ajustada) y polígonos aproximados, además de la línea amarilla que delimita la última fila. La tabla confirma la clasificación: circularidades 0.89–0.92 y solidity 0.99 para la mayoría de círculos; ratios de ejes altos (p. ej. $AR=1.99$ y $AR=3.26$) para las

elipses; y recuento de lados coherente para figuras tipo triángulo/cuadrado/hexágono (3/4/6). Para los pacman (blobs 17–20) se estima la cuña faltante con valores decrecientes (23.6%, 15.0%, 5.9%, 4.4%), consistente con que el mordisco se reduce de izquierda a derecha. Como reflexión mínima del enunciado: este enfoque funciona bien porque usa métricas geométricas simples (circularidad/solidity/axis ratio), pero es sensible a la segmentación (umbral y morfología) y a la elección de epsilon en approxPolyDP; si el contorno se degrada o hay ruido, la circularidad, el número de vértices y el porcentaje de cuña pueden sesgarse.

Ejercicio 3.b

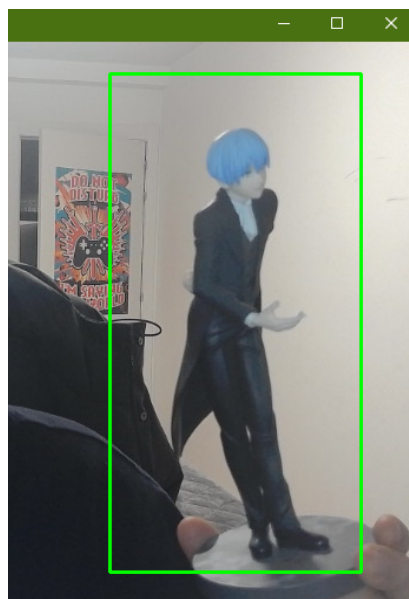
En este ejercicio se implementa detección de caras en webcam en tiempo real con Haar en cascada, combinando un detector frontal y otro de perfil para mejorar cobertura cuando la cabeza no está completamente de frente. El flujo es: captura de vídeo, paso a gris y ecualización de histograma (equalizeHist) para estabilizar contraste, ejecución de detectMultiScale con parámetros de escala y vecindad (scaleFactor, minNeighbors, minSize) tanto para frontal como perfil, y fusión de ambas listas de detecciones. Para evitar duplicados (frontal+perfil detectando la misma cara), se aplica una supresión por solape tipo NMS basada en IoU con umbral IOU_SUPPRESS_TH, y finalmente se dibuja el bounding box y se imprime periódicamente el número de caras y el tiempo de detección.



En el resultado se detecta correctamente la cara del retrato, con una caja estable y bien ajustada, lo que indica que la combinación de ecualización + cascada frontal/perfil funciona en una condición favorable (cara relativamente grande, con rasgos contrastados). Como reflexión mínima pedida: este método suele degradarse con cambios de escala (caras pequeñas/lejos), rotaciones fuera de plano, oclusiones (mano/mascarilla) e iluminación irregular/contraluz; en tu implementación, el uso de dos cascadas aumenta la probabilidad de detección ante giros laterales, y el NMS por IoU ayuda a controlar duplicados, pero no evita falsos positivos si el fondo contiene patrones tipo cara.

Ejercicio 3.c

En este ejercicio se implementa detección de peatones en webcam usando HOGDescriptor + SVM preentrenado de OpenCV. El flujo es: captura de cámara, redimensionado para limitar coste (MAX_WIDTH=640), detección con `hog.detectMultiScale` ajustando `winStride`, `padding` y `scale`, y filtrado opcional por score (en este caso el umbral MIN_WEIGHT es muy bajo, así que prácticamente se aceptan todas). Como el detector suele producir múltiples cajas solapadas para la misma persona, se aplica una Non-Max Suppression propia basada en ratio de solape (NMS_OVERLAP_TH=0.65) para consolidar una única detección final por individuo. También se imprime cada cierto tiempo el número de cajas antes/después de NMS y el tiempo de cómputo para valorar rendimiento.



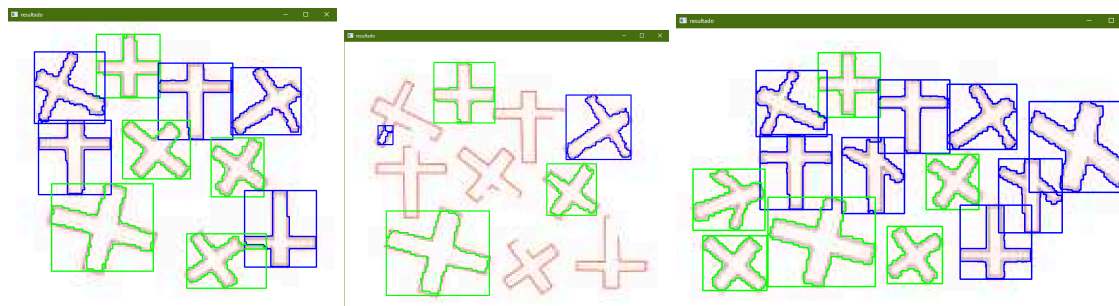
En este resultado, tras aplicar HOG+SVM y la supresión de no-máximos (NMS), queda una caja final en verde que engloba de forma coherente la figura completa (cuerpo entero), lo que confirma que el pipeline `detectMultiScale` + NMS está consolidando correctamente las hipótesis redundantes en una detección estable. Además, tal y como has comprobado en tus pruebas, y al igual que en 3b, el detector se comporta de forma consistente no solo con personas reales, sino también con personas de diferentes etnias y con figuras humanoides, manteniendo detecciones razonables siempre que la silueta sea similar a un peatón de cuerpo entero (postura y escala dentro del rango esperado). La robustez depende más de condiciones de captura (tamaño de cara/cuerpo, rotación, oclusión e iluminación) que del individuo en sí.

Ejercicio 4

Se diseña un sistema que localice y clasifique cruces griegas vs latinas, rechazando objetos con brazos no perpendiculares, en cuatro escenarios: (a) ideal, (b) contornos incompletos, (c) defectos, y (d) iluminación irregular + ruido.

Ejercicio 4.a-b-c

En este ejercicio se pide localizar y clasificar cruces en griegas (simétricas, tipo X) y latinas (con un eje claramente dominante, tipo T o + descompensada), y hacerlo en tres escenarios con dificultad creciente (ideal, contornos incompletos y defectos). Se aplica un pipeline común para a/b/c: primero se mejora el contraste con CLAHE y se binariza con Otsu, eligiendo automáticamente entre modo normal o invertido según el porcentaje de foreground esperado (FG_MIN..FG_MAX), para evitar inversiones cuando cambia el fondo. Después se refuerza la máscara con dilatación y morfología (close/open) y se rellenan huecos para solidificar las regiones. Para separar cruces tocándose, se usa distance transform para generar semillas y se aplica watershed, quedándose con etiquetas de objetos. Finalmente, para cada ROI se clasifica por análisis de ejes principales apoyado en distance transform (ponderando más las zonas centrales del trazo): se calculan dos direcciones principales y se estiman sus longitudes por raycast dentro del objeto; con el ratio L_{max}/L_{min} se decide latina si supera $RATIO_LATINA_MIN=1.25$ y griega si no.

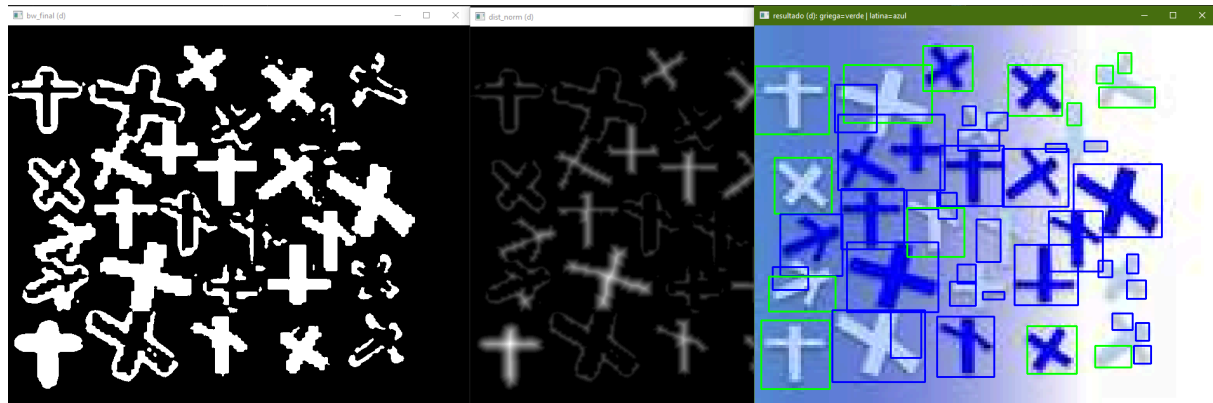


En los resultados se observa una detección y clasificación consistente: las cruces tipo X quedan marcadas en verde (griegas) y las cruces tipo T o con un eje predominante quedan en azul (latinas), manteniendo el comportamiento incluso cuando hay contornos incompletos (b) o irregularidades/defectos/ (c). En el caso con más densidad de objetos (c) se aprecia que watershed ayuda a separar instancias próximas, aunque aparecen algunos solapes de cajas o recortes ajustados al bounding box cuando las cruces están muy juntas o el contorno está degradado. El criterio por ratio de ejes es adecuado para distinguir simetría frente a eje dominante, pero su punto débil son deformaciones que alteran el centro del objeto o rompen brazos (porque sesgan el eje principal) y, si se quiere reforzar el rechazo de brazos no perpendiculares, una mejora mínima sería añadir una comprobación explícita de ortogonalidad entre ejes (producto escalar cercano a 0) y descartar/etiquetar como no válida cuando se aleje claramente de 90° , pero la única forma que medianamente funcionó a la hora de la práctica, fue la planteada en código.

Ejercicio 4.d

En este escenario se trabaja con cruces bajo ruido e iluminación irregular, por lo que se prioriza una segmentación robusta sin depender de umbral global. Se aplica un denoising con fastNlMeansDenoising (parámetro DENOISE_H), seguido de un umbral adaptativo gaussiano en invertido (ADAPTIVE_BLOCK y ADAPTIVE_C) para compensar variaciones de fondo. A continuación se

engrosa el trazo con dilatación, se consolidan regiones con cierre morfológico, se rellenan huecos mediante floodFill (fill_holes_imfill) y se limpia con apertura. Los objetos se extraen con connectedComponentsWithStats filtrando por área, y cada ROI se solidifica (close + fill holes) antes de calcular ejes principales ponderados por distance transform (DT_WEIGHT_POWER). La clasificación se realiza con el ratio de longitudes de ejes L_{max}/L_{min} , etiquetando como latina si supera $RATIO_LATINA_MIN=1.25$ y como griega en caso contrario.



En el resultado se observa que, pese a la degradación severa, se obtiene una máscara binaria final medio utilizable y un mapa de distance transform que resalta las zonas centrales de los trazos, permitiendo clasificar algunas cruces: las griegas quedan marcadas en verde y las latinas en azul. También se aprecia la presencia de múltiples cajas pequeñas (falsos positivos) en zonas de ruido residual, lo que indica que el umbral adaptativo y la morfología recuperan estructura pero no eliminan completamente fragmentos. Como reflexión, este caso confirma que con iluminación irregular el umbral adaptativo es más adecuado que uno global, pero la robustez final depende de controlar el ruido mediante filtrado por área y, si fuese necesario, añadir criterios extra de validación de forma (por ejemplo, descartar ROIs con ratio extremo por fragmentación o comprobar ortogonalidad aproximada de ejes para rechazar no-cruces). O directamente haber hallado un mejor método para aislar las cruces en un principio, aunque de lo que probé esto fue lo que mejor resultado me dio.